

Migration Experiment_00-data-prep

Setup

Libraries and Load Data

Quick Fixes

Taxonomy Update to Massachusetts Breeding Lists

```
# Taxonomy Updates!
MA_status <- MA_status_raw %>%
  mutate(species_code = replace_when(species_code,
    species_code == "yelwar" ~ "yelwar1",
    species_code == "warvir" ~ "eawvir1"
  ))
```

Resident Tag

```
# Resident vs Non-resident Column
MA_status <- MA_status %>%
  mutate(
    Resident = case_when(
      max_week_season == "resident" ~ TRUE,
      max_week_season != "resident" ~ FALSE
    )
  )
```

Drop Totally Unneeded columns

- Leave Scientific Name for Auk Rollup

```
# Drop myebird_EXP columns #####
myebird_EXP <- myebird_raw %>%
  select(-Area.Covered..ha.,
    -Number.of.Observers, -Breeding.Code,
    -ML.Catalog.Numbers, -State.Province, -Distance.Traveled..km., -Duration..Min.)

# Drop taxonomy columns #####
ebird_taxonomy <- ebird_taxonomy_raw %>%
  select(-CATEGORY, -TAXON_CONCEPT_ID, -REPORT_AS)
```

```
# Hold onto max_week_season until making Resident column
MA_status <- MA_status %>%
  select(species_code, common_name, scientific_name, Resident)
```

Renaming columns and protocol type

- Wait to drop columns bc some are needed for auk cleaning

```
myebird_EXP <- myebird_EXP %>% mutate(
  Protocol = case_when(
    grepl("Traveling", Protocol) ~ "Traveling",
    grepl("Casual", Protocol) ~ "Incidental",
    grepl("Stationary", Protocol) ~ "Stationary")
) %>%
  rename("Is.Complete" = "All.Obs.Reported",
        "Species" = "Common.Name",
        "Taxonomy" = "Taxonomic.Order",
        "Checklist.ID" = "Submission.ID")
```

Auk Rollup

Auk Rollup purpose: remove unidentifiable species (hybrids, sp.) and roll-up subspecies

Note on “sp.”: short for “species”, used for sightings where species can’t be determined beyond a genus or broad grouping (eg. sea gull sp., Empidonax sp.)

```
# RENAME COLUMNS temporarily to match auk
myebird_EXP_rolled <- rename(myebird_EXP,
  checklist_id = Checklist.ID,
  scientific_name = Scientific.Name,
  observation_count = Count)

# Rollup!
myebird_EXP_rolled <- auk_rollup(myebird_EXP_rolled)

# SWITCH COLUMN NAMES back and RESAVE as myebird
myebird_EXP <- myebird_EXP_rolled %>%
  rename(Checklist.ID = checklist_id,
        Scientific.Name = scientific_name,
        Count = observation_count) %>%
  select(-c("taxonomic_order", "taxon_concept_id"))

# Drop Column Scientific Name now that we're done with it
myebird_EXP <- myebird_EXP %>%
  select(-Scientific.Name)
rm(myebird_EXP_rolled)
```

Add Is.Paired Column

Some days I did only one timepoint, or went at a different time. For example, one morning it started raining halfway through my dawn timepoint, so I cancelled the second timepoint and decided to exclude that day from my analysis.

This step is taking place before **before** building widening the myebird dataframe, so that only species sighted during the actual experiment are included downstream.

```
paired_lookup <- tribble(
  ~Checklist.ID, ~Is.Paired,
  "CHK-0011", 1,
  "CHK-0012", 1,
  "CHK-0013", 1,
  "CHK-0014", 1,
  "CHK-0015", 1,
  "CHK-0016", 1,
  "CHK-0017", 0,
  "CHK-0018", 1,
  "CHK-0019", 1,
  "CHK-0020", 1,
  "CHK-0021", 1,
  "CHK-0022", 0,
  "CHK-0025", 1,
  "CHK-0026", 1,
  "CHK-0023", 1,
  "CHK-0027", 1,
  "CHK-0024", 1,
  "CHK-0028", 1,
  # New entries
  "CHK-0029", 0,
  "CHK-0030", 1,
  "CHK-0031", 0,
  "CHK-0032", 1,
  "CHK-0033", 1,
  "CHK-0034", 1,
  "CHK-0036", 1,
  "CHK-0037", 1,
  "CHK-0035", 0,
  "CHK-0038", 1,
  "CHK-0039", 1,
  "CHK-0040", 0,
  "CHK-0041", 0,
  # 5/1 single timepoint, no pair -- must be 0 or it's dropped from both EXP and extra_obs
  "CHK-0010", 0,
  # Outside the May-timepoint window entirely (not a rained-out pair day, just
# a sighting outside the experiment's date range) -- kept as candidates for
# the "red asterisk" extra-observation markers in the 01 file. See the
# Presence.Only flag below for which of these can actually use their count.
  "CHK-0004", 0,
  "CHK-0005", 0,
  "CHK-0006", 0,
  "CHK-0007", 0,
  "CHK-0008", 0,
  "CHK-0009", 0
)

# JOIN TO MY_EBIRD!
myebird_EXP <- left_join(myebird_EXP, paired_lookup, by = "Checklist.ID") #>%
  # relocate(Is.Paired, .after = Is.Complete)
```

Flag Long-Route Extra Sightings as Presence-Only

Some of the extra (`Is.Paired == 0`, outside the May-timepoint window) checklists above covered more ground/time than my actual experiment protocol – a normal experiment checklist here runs about 0.8-1.07 km and 50-73 min. A longer route sees more birds just by virtue of covering more ground, so its Count would be biased toward HIGHER numbers – not a fair value to plot as a real “extra sighting” count yet. Any of these longer than **1 km OR 1 hour** get flagged `Presence.Only = TRUE`, so the 01 file can drop them from the count-based red asterisks for now (they can come back later once there’s a real presence-only marker style for them) while keeping every sighting itself in the data.

`Presence.Only` only makes sense as a concept for `myebird_EXP_extra_obs` (the red-asterisk candidates) – the main `myebird_EXP` is only ever the real paired protocol checklists, so it doesn’t get this column at all. The join happens below, once `myebird_EXP_extra_obs` actually exists.

Computed from `myebird_raw`, not hardcoded – `Distance.Traveled..km.` and `Duration..Min.` get dropped from `myebird_EXP` above before `Checklist.ID` even exists, but `myebird_raw` itself is never touched, so the real numbers are still sitting there under their original column names.

```
presence_only_lookup <- myebird_raw %>%
  filter(Submission.ID %in% c("CHK-0004", "CHK-0005", "CHK-0006",
                             "CHK-0007", "CHK-0008", "CHK-0009")) %>%
  distinct(Checklist.ID = Submission.ID, Distance.Traveled..km., Duration..Min.) %>%
  mutate(Presence.Only = Distance.Traveled..km. > 1 | Duration..Min. > 60)
```

`presence_only_lookup` *# printed so the flag stays auditable against real Distance/Duration, not a black*

```
## Checklist.ID Distance.Traveled..km. Duration..Min. Presence.Only
## 1      CHK-0006           1.624           60           TRUE
## 2      CHK-0009           1.744           58           TRUE
## 3      CHK-0007           1.747           59           TRUE
## 4      CHK-0005           1.510           71           TRUE
## 5      CHK-0008           1.802           52           TRUE
## 6      CHK-0004           1.782           56           TRUE
```

DATA CLEANING

NA and 0's

Check for NAs

```
which(is.na(myebird_EXP))
```

```
## integer(0)
```

```
sum(is.na(myebird_EXP))
```

```
## [1] 0
```

for Stationary and Incomplete Checklists

Incidental checklists should already all be marked incomplete, but forcing Is.Complete to 0 just to be safe

```
myebird_EXP <- myebird_EXP %>% mutate(  
  Is.Complete = case_when(  
    grepl("Incidental", Protocol) ~ 0,  
    TRUE ~ Is.Complete  
  )  
)
```

Rearrange Columns

```
# Rearrange columns  
myebird_EXP <- relocate(myebird_EXP, Protocol, .after = Checklist.ID)  
myebird_EXP <- relocate(myebird_EXP, Is.Complete, .after = Protocol)  
myebird_EXP <- relocate(myebird_EXP, Is.Paired, .after = Is.Complete)
```

Dates

Make Year, Month, Day columns

```
# Convert to date format  
myebird_EXP$Date <- as.Date(myebird_EXP$Date)  
# Extract year, month, day (lubridate package)  
myebird_EXP <- myebird_EXP %>%  
  mutate(  
    Year = year(Date),  
    Month = month(Date),  
    Day = day(Date),  
    .after = Date)  
  
# Update data types  
myebird_EXP <- myebird_EXP %>%  
  mutate(across(c(Year, Month, Day), as.integer))
```

Add Month.Day column for graph axis labels

```
myebird_EXP <- myebird_EXP %>% mutate(  
  Month.Day = factor(  
    format(Date, "%b %d"),  
    levels = format(sort(unique(Date)), "%b %d")  
  ),  
  .after = Date  
)
```

Time

- Time format is “06:22 PM”

```

# convert time from <chr> format to <dtm>
myebird_EXP <- myebird_EXP %>%
  mutate(
    Time = strptime(myebird_EXP$Time, format = "%I:%M %p")

# use dtm formatted Time to get Hour and Minute, use to calculate time as a <dbl>, then drop Hour and Minute
# Create Start.Time
# Drop dtm "Time" (will use Start.Time instead), Hour and Minute

myebird_EXP <- myebird_EXP %>%
  mutate(
    Start.Time = hour(Time) + minute(Time)/60,
    Hour = hour(Time),
    Minute = minute(Time),
    .after = Time
  ) %>%
  select (-Time, -Hour, -Minute)

```

FILTER CHECKLISTS HERE !! COMPLETE/INCOMPLETE, PAIRED/UNPAIRED, DATE START HERE

Filter

- Main analysis “EXP” Experiment includes ONLY complete, paired checklists
- “Extra observations” will be used for some graphs
- Experiment began after May 1st

```

# must generate extra_obs before overwriting myebird_EXP
myebird_EXP_extra_obs <- myebird_EXP %>% filter(Is.Complete == 0 | Is.Paired == 0 | Date < as.Date("2026-05-01"))
myebird_EXP <- myebird_EXP %>% filter(Is.Complete == 1, Is.Paired == 1, Date > as.Date("2026-05-01"))

# Presence.Only only ever applies to the extra/red-asterisk candidates above --
# myebird_EXP (the real paired protocol data) never gets this column
myebird_EXP_extra_obs <- myebird_EXP_extra_obs %>%
  left_join(select(presence_only_lookup, Checklist.ID, Presence.Only), by = "Checklist.ID") %>%
  mutate(Presence.Only = replace_na(Presence.Only, FALSE))

```

Save basic stats and counts for EXP and EXP_extra_obs

```

# SPECIES AND SPECIES COUNTS

species <- unique(myebird_EXP$Species)
species_extra <- unique(myebird_EXP_extra_obs$Species)
species_all <- c(species, species_extra) %>% unique()
# species seen in unpaired/incomplete checklists ("extra") but NOT in regular experiment
log_filter <- !(species_extra %in% species)
species_extra_only <- species_extra[log_filter]

num_species <- length(species)

```

```
num_species_extra <- n_distinct(myebird_EXP_extra_obs$Species)
num_species_all <- n_distinct(species_all)
```

```
# NUMBER OF CHECKLISTS
```

```
num_checklists <- n_distinct(myebird_EXP$Checklist.ID)
num_checklists_extra <- n_distinct(myebird_EXP_extra_obs$Checklist.ID)
```

Quality Check

```
# How many of each Protocol is there?
```

```
cat("First myebird_EXP:\n")
```

```
## First myebird_EXP:
```

```
myebird_EXP %>% distinct(Checklist.ID,Protocol) %>% count(Protocol)
```

```
## # A tibble: 1 x 2
##   Protocol      n
##   <chr>      <int>
## 1 Traveling    24
```

```
cat("Then myebird_EXP_extra_obs:\n")
```

```
## Then myebird_EXP_extra_obs:
```

```
myebird_EXP_extra_obs %>% distinct(Checklist.ID,Protocol) %>% count(Protocol)
```

```
## # A tibble: 2 x 2
##   Protocol      n
##   <chr>      <int>
## 1 Incidental     4
## 2 Traveling    10
```

```
# NA Check
```

```
cat("NA's in myebird_EXP (there should be NONE):\n\n")
```

```
## NA's in myebird_EXP (there should be NONE):
```

```
colSums(is.na(myebird_EXP))
```

```
## Checklist.ID    Protocol  Is.Complete  Is.Paired    Species    Taxonomy
##           0           0           0           0           0           0
##      Count    Location      Date    Month.Day      Year      Month
##           0           0           0           0           0           0
##      Day    Start.Time
##           0           0
```

```
cat("\n\n\nNA's in myebird_EXP_extra_obs (there should be NONE):\n\n")
```

```
##
##
##
## NA's in myebird_EXP_extra_obs (there should be NONE):
```

```
colSums(is.na(myebird_EXP_extra_obs))
```

```
## Checklist.ID      Protocol  Is.Complete  Is.Paired    Species
##           0           0           0           0           0
##      Taxonomy      Count      Location        Date      Month.Day
##           0           0           0           0           0
##          Year      Month        Day    Start.Time Presence.Only
##           0           0           0           0           0
```

JOIN TAXONOMY and RESIDENT STATUS DATA

Taxonomy and Population lists

```
# Taxonomy and Species, of all species in my experiment, ordered by Taxonomy
my_taxonomy_EXP <- myebird_EXP %>%
  distinct(Species, Taxonomy) %>%
  arrange(Taxonomy)

# Vector of Species Order
species_order <- my_taxonomy_EXP$Species

#####
# ADD EBIRD'S TAXONOMY TO my_taxonomy_EXP! (species code, species group)

# 1) Change ebird_taxonomy column names to match my_tax
# Change any other column names (species code)
# Keep species code for matching to MA data
ebird_taxonomy <- ebird_taxonomy %>% rename(
  Taxonomy = TAXON_ORDER,
  Species_unrolled = PRIMARY_COM_NAME,
  Species.Group = SPECIES_GROUP,
  Species.Code = SPECIES_CODE
)

# 2) Add SPECIES GROUP from ebird_tax to my_tax
# DONT include species in ebird_tax that aren't also in my_tax (left_join)
my_taxonomy_EXP <- left_join(my_taxonomy_EXP, ebird_taxonomy, by = "Taxonomy") %>%
  select(Species, Taxonomy, Species.Code, Species.Group)

# 3) Shorten Species.Group names
# I'm turning "Species.Group" into a grouping of convenience for graphing, not an accurate phylogenetic
my_taxonomy_EXP <- my_taxonomy_EXP %>%
  mutate(
```

```

Species.Group = replace_when(Species.Group,
  Species.Group == "Cormorants and Anhingas" ~ "Cormorants",
  Species.Group == "Herons, Ibis, and Allies" ~ "Herons",
  Species.Group == "Tits, Chickadees, and Titmice" ~ "Chickadees and Titmice",
  Species.Group == "Tyrant Flycatchers: Pewees, Kingbirds, and Allies" ~ "Flycatchers",
  Species.Group == "Vultures, Hawks, and Allies" ~ "Hawks and Vultures",
  Species.Group == "Jays, Magpies, Crows, and Ravens" ~ "Corvids",
  Species.Group == "Starlings and Mynas" ~ "Starlings",
  Species.Group == "Finches, Euphonias, and Allies" ~ "Finches",
  Species.Group == "New World Sparrows" ~ "Sparrows",
  Species.Group == "Wood-Warblers" ~ "Warblers",
  Species.Group == "Cardinals, Grosbeaks, and Allies" ~ "Cardinal Family"
))

#####
# ADD MA STATS TO my_taxonomy_EXP! (breeding %, max pop season/week, etc)

# 1) Make sure join column matches
MA_status <- MA_status %>% rename(
  Species.Code = species_code
)

# 2) Join by Species.Code
my_taxonomy_EXP <- left_join(my_taxonomy_EXP, MA_status, by = "Species.Code") %>%
  select(-common_name) %>%
  relocate(Taxonomy, .before = Species) %>%
  relocate(Species.Code, .before = Species) %>%
  mutate(Scientific.Name = scientific_name, .after=Species) %>%
  select(-scientific_name)

```

WIDE FORM

Make Wide Version

Now make our checklist-wise, widened df, with new column PER SPECIES! (equivalent to “checklists” + each species)

```

# Taxonomy varies by species, so keep only checklist-level fields as pivot identifiers
checklist_metadata <- myebird_EXP %>%
  select(-Species, -Count, -Taxonomy) %>%
  distinct()

checklists_EXP <- myebird_EXP %>%
  select(Checklist.ID, Species, Count) %>%
  pivot_wider(
    id_cols = Checklist.ID,
    names_from = Species,
    values_from = Count
  ) %>%
  left_join(checklist_metadata, ., by = "Checklist.ID") %>%
  select(all_of(names(checklist_metadata)), all_of(my_taxonomy_EXP$Species)) %>%
  mutate(across(all_of(my_taxonomy_EXP$Species), as.integer))

```

```

expected_num_checklists <- 24L
stopifnot(
  "checklists_EXP must have one row per Checklist.ID" =
    !anyDuplicated(checklists_EXP$Checklist.ID),
  "checklists_EXP must contain exactly 24 experiment checklists" =
    nrow(checklists_EXP) == expected_num_checklists
)

```

Zero-Fill

```

# Replace species count NAs with 0s
# must use across() to apply replace_na inside mutate, because mutate goes one col at a time

checklists_EXP <- checklists_EXP %>%
  mutate(
    across(all_of(my_taxonomy_EXP$Species), ~replace_na(.,0))
  )

```

Binning

Start Time Bins

```

checklists_EXP <- checklists_EXP %>%
  mutate(
    Start.Time.Bin = factor(
      case_when(
        Start.Time < 8.0 ~ "Dawn",
        Start.Time < 11.0 ~ "Morning",
        Start.Time < 13.0 ~ "Mid-Day",
        levels = c("Dawn", "Morning", "Mid-Day"),
        ordered = TRUE),
    .after = Start.Time)
  )

```

PIVOT BACK TO MYEBIRD LONG

Make new Long-form DF

- Make a new long-form (species-wise on each row) so I can feed Species column to the heatmap y-axis
- Use the wide df (checklist-wise) so that the new long-form has zero-filled species and all other new checklist-specific cols

```

myebird_EXP <- checklists_EXP %>%
  pivot_longer(
    cols = all_of(my_taxonomy_EXP$Species),
    names_to = "Species",
    values_to = "Count"
  )

```

Turn Species column into an ordered factor (so species are sorted automatically by taxonomy)

```
myebird_EXP <- myebird_EXP %>%  
  mutate(Species = factor(Species, levels = my_taxonomy_EXP$Species))
```

Quality Check

Confirm number of rows is correct! - myebird_EXP now has 1 row per species per experiment, covering ALL SPECIES seen in ENTIRE EXPERIMENT, not just that one checklist - `nrow(myebird_EXP)` should = #species * # checklists

```
# Number of Species in Experiment Overall!!
```

```
num_species <- n_distinct(species_order)
```

```
num_checklists<- n_distinct(myebird_EXP$Checklist.ID)
```

```
# Number of species and checklists
```

```
cat("This experiment dataset contains: ", num_species, "Species,", num_checklists, "Checklists\n\n")
```

```
## This experiment dataset contains: 72 Species, 24 Checklists
```

```
# Number of rows in myebird_EXP
```

```
cat("In myebird_EXP (long form): there's one row per species (all species from WHOLE EXPERIMENT) per check
```

```
## In myebird_EXP (long form): there's one row per species (all species from WHOLE EXPERIMENT) per check
```

```
cat("Number of Rows should equal 'num_species * num_checklists': ", nrow(myebird_EXP), "Rows,", num_spe
```

```
## Number of Rows should equal 'num_species * num_checklists': 1728 Rows, 1728 Calculation
```

```
#nrow(myebird_EXP)
```

```
#num_species*num_checklists
```

SUPPLEMENTARY OUT-OF-PROTOCOL OBSERVATIONS

```
# Detections only (NOT zero-filled)
```

```
# Tag WHY each point is out-of-protocol (drives the caption, and lets you filter)
```

```
exp_start <- min(myebird_EXP$Date)
```

```
exp_end <- max(myebird_EXP$Date)
```

```
myebird_EXP_extra_obs <- myebird_EXP_extra_obs %>%
```

```
  mutate(protocol_note = case_when(
```

```
    Date < exp_start ~ "Pre-experiment (route/timing still varying)",
```

```
    Date > exp_end ~ "Post-experiment",
```

```
    TRUE ~ "During-experiment, unpaired or incomplete"
```

```
  ))
```

Save Dataframes!

```
# Make sure date stays as date type
myebird_EXP <- mutate(myebird_EXP,
                      Date = as.Date(myebird_EXP$Date))
checklists_EXP <- mutate(checklists_EXP,
                          Date = as.Date(checklists_EXP$Date))
myebird_EXP_extra_obs <- mutate(myebird_EXP_extra_obs,
                                Date = as.Date(myebird_EXP_extra_obs$Date))

# Save Dataframes as RDS to conserve datatypes!!
# For use in analysis file!

# myebird
saveRDS(myebird_EXP, file = "./data/derived/myebird_EXP.rds")

# myebird_extra_obs
saveRDS(myebird_EXP_extra_obs, file = "./data/derived/myebird_EXP_extra_obs.rds")

# ebird_taxonomy
saveRDS(ebird_taxonomy, file = "./data/derived/ebird_taxonomy_EXP.rds")

# my_taxonomy
saveRDS(my_taxonomy_EXP, file = "./data/derived/my_taxonomy_EXP.rds")

# checklists
saveRDS(checklists_EXP, file = "./data/derived/checklists_EXP.rds")
```